

Scalable Heterogeneous Scheduling Based Model Parallelism for Real-Time Inference of Large-Scale Deep Neural Networks

Xiaofeng Zou¹, Cen Chen¹, Senior Member, IEEE, Peiying Lin², Luochuan Zhang¹, Yanwu Xu¹, Senior Member, IEEE, and Wenjie Zhang³, Member, IEEE

Abstract—Scaling up the capacity of deep neural networks (DNN) is one of the effective approaches to improve the model quality for several different DNN-based applications, making the DNN models continuously grow. To promote the execution efficiency of large and complex models, the devices are becoming increasingly heterogeneous with CPUs and domain-specific hardware accelerators. In many cases, the capacity of large-scale models is beyond the memory limit of a single accelerator. Recent work has shown that model parallelism, which aims to partition a DNN's computational graph on multiple devices, can not only address this problem while also provide significant performance improvements. In this work, we focus on optimizing model parallelism for timely inference of large-scale DNNs on heterogeneous processors. We transform the computation graphs of DNNs into directed acyclic graphs (DAGs) and propose to utilize heterogeneous scheduling methods to determine the model partition plan. Nevertheless, we have found that current efficient DAG scheduling methods have a lot of room for improvement to process large-scale DAGs and have high computation complexity. To this end, we propose a scalable DAG partition assisted scheduling method for heterogeneous processors to address these problems. Our approach takes the execution time of DNN models, high scalability, and memory constraints into consideration. We demonstrate the effectiveness of our approaches using both small- and large-scale DNN models. To the best of our knowledge, it is the first work that explores DAG scheduling and partitioning methods for model parallelism, and provides new avenues for accelerating large-scale DNN inference.

Index Terms—Deep neural networks, model parallelism, parallel computing.

Manuscript received 19 July 2023; revised 15 October 2023; accepted 12 December 2023. Date of publication 7 March 2024; date of current version 24 July 2024. This work was supported in part by the National Funding Program for Postdoctoral Researchers under Grant GZC20230841, in part by the Fundamental Research Funds for the Central Universities under Grant x2wjD2230230, in part by the Natural Science Foundation of Guangdong Province of China, and in part by CCF-Phytium Fund and CAAI-MindSpore Open Fund. (Corresponding author: Cen Chen.)

Xiaofeng Zou and Luochuan Zhang are with the School of Future Technology, South China University of Technology, Guangzhou 510641, China (e-mail: zouxiaofeng@hnu.edu.cn; zlc_0704@126.com).

Cen Chen and Yanwu Xu are with the School of Future Technology, South China University of Technology, Guangzhou 510641, China, and also with Pazhou Laboratory, Guangzhou 510330, China (e-mail: chencen@scut.edu.cn; ywxu@ieee.org).

Peiying Lin is with the School of Computer Science & School of Cyberspace Science, Xiangtan University (XTU), Xiangtan 411105, China (e-mail: peiying_lin@hnu.edu.cn).

Wenjie Zhang is with the Department of Electrical Engineering, The Hong Kong Polytechnic University, Kowloon, Hong Kong (e-mail: wenjie_zhang@u.nus.edu).

Recommended for acceptance by J. T. Zhou.

Digital Object Identifier 10.1109/TETCI.2024.3369628

I. INTRODUCTION

DEEP neural networks (DNNs) have demonstrated substantial successes in various domains, including image recognition [1], [2], [3] and natural language processing (NLP) [4], [5], [6]. Currently, the widely utilized DNN models are generally complex and large-scale. For instance, the popular DNN model VGG-16 [7], which is utilized for computer vision, has up to 138 million parameters. BERT-24 [8], which has obtained remarkable results in NLP tasks, has up to 340 million parameters. Scaling up the capacity of DNN models is an effective approach to improve the model performance for complex tasks. Following such successes, some even larger models are proposed, including T5 [9] (11 billion parameters), GPT-3 [4] (175 billion parameters), and GShard [10] (600 billion parameters). These large-scale models often exceed the memory capabilities of accelerator devices (e.g., GPUs). To address this limitation, techniques like model parallelism techniques have been proposed. These techniques [11], [12] partition a DNN's computational graph onto multiple devices for parallel execution. Recent work has shown that model parallelism can effectively address the limitation while also achieving significant performance improvements [13].

In industrial settings, the ability to perform real-time inference is essential. For example, the real-time inference of the text-to-speech (TTS) system is very important [14]. Another typical application is autonomous driving, where delayed inference could cause disastrous problems. Model parallelism, which involves partitioning models and enabling parallel inference on multiple devices, is a promising method to enhance inference performance.

It has been established that determining the optimal partition of DNN models is NP-hard [15], especially for large-scale DNN models. Most previous work on automated model partition can be broadly categorized into two groups: learning based methods [16], [17], [18], [19] and cost model based methods [20], [21], [22]. Detailed descriptions of these works are presented in Section II-A. Despite significant research efforts directed toward optimizing model parallelism for the training throughput of DNNs, comparatively limited attention has been paid to the timely execution of DNN inference via model parallelism.

The automated DNN model partition methods currently available are computationally expensive. For the learning-based

methods, it is necessary to evaluate large amounts of placements, where each one requires reconfiguring deployed devices (for new DNN separation) and measuring the runtime of multiple training/inference stages. For example, [16] needs 12 – 27 hours of training time to partition modern workloads and [17] requires 12 hours execution on a GPU. For the cost model-based methods, current state-of-the-art methods are also computationally expensive to search the partition solutions of DNNs. For instance, a recent state-of-the-art work for model partition [15], which exploits the dynamic programming method to search for the best partition of the model, has high computational complexity during the searching process. It requires 13 hours to find a relatively optimal partition of BERT-12 model which only has 783 operation nodes. As previously discussed, modern DNN models are large-scale and continuously growing. The running time and the scalability of the partition algorithm are also critical factors to consider.

Popular computing frameworks, such as TensorFlow, PyTorch, and MxNet, accomplish the design and implementation of a DNN by constructing and executing operation computing graphs which are directed acyclic graphs (DAGs). Recent extensive branching in DNN architecture, along with operator-granularity DAG for established DNNs, offer the potential to execute data-independent parts of the computation in parallel to reduce latency. There are many DAG scheduling methods [23], [24] which aim to assign the tasks of an application to appropriate processors to minimize the execution time, or *makespan*, of DAG tasks. These methods have been successful in scheduling tasks.

Motivated by these observations, we propose a scalable automated model partition method, termed as SAMP, for real-time inference of large-scale DNNs on heterogeneous processors based on some core ideas learned from current heterogeneous DAG scheduling methods. There are three main processes in SAMP: latency-driven multilevel DAG partition, DAG partition scheduling on heterogeneous processors, and DNN model partition based on the scheduling results. (i) The latency-driven multilevel DAG partition aims to cluster operators in DNNs to enhance the data locality with a global view of the graph execution latency. The DAG could also be reduced during the DAG partition phase. Moreover, since the partitions are acyclic, it is possible to schedule each partition completely once its input task is ready. (ii) Based on the partitioned results, we utilize our proposed DAG partition scheduling method to schedule the partitioned DAG on heterogeneous processors. The objective of this phase is to minimize the execution time, or *makespan*, of the partitioned DAG. After scheduling, the DNN model can be executed on heterogeneous processors in parallel. (iii) Based on the scheduling results, operations that are scheduled on the same processor are grouped together as a partition. In the meanwhile, the model parameters are partitioned according to the operation partition results.

The proposed method, SAMP, has two main objectives: minimize the execution time of DNN models, and high scalability. The latency-driven multilevel DAG partition and DAG partition scheduling phases take both the execution time of DNNs and the memory constraints of accelerators into consideration.

Moreover, after the first DAG partition phase, the operation DAG of DNN is reduced, thus reducing the search space of the second scheduling phase. This mechanism enables SAMP to be able to process large-scale DNNs.

Experimental results indicate that SAMP is efficient for processing small, medium, and large models and has low computation complexity. Specifically, SAMP obtains more than $20\times$ improvement compared with the latest state-of-the-art model partition methods IP [15] on average, obtains more than $3\times$ improvement compared with the popular heterogeneous scheduling method (HEFT) [23]. Moreover, SAMP can process large-scale models in a few seconds. The running time is much shorter than that of IP which requires more than 10 hours to process. SAMP is also more than $100\times$ faster compared with HEFT for processing large-scale models,

The main contributions of this work are summarized as follows.

- 1) We propose a scalable automated model partition method, SAMP, for real-time inference of large-scale DNNs on heterogeneous processors by exploring ideas in heterogeneous DAG scheduling methods.
- 2) SAMP is a hybrid DAG partition and DAG scheduling method, aiming to minimize the execution time of DNN models by model parallelism with memory constraints and low computation complexity.
- 3) We demonstrate the applicability, efficiency, and scalability of our approaches using both small and large-scale DNN models.

In contrast to current model partition methods, which primarily focus on improving the throughput of DNN training utilizing model parallelism, SAMP targets the real-time inference of large-scale DNNs. SAMP opens new opportunities for exploring DAG scheduling ideas for the acceleration of DNN inference.

II. RELATED WORK

A. Model Parallelism of DNN

Model parallelism and data parallelism are two crucial methods for achieving parallelism in DNN [21]. Data parallelism improves execution efficiency by partitioning data and distributing it to multiple devices/processors for parallel execution [25]. However, this approach requires each processor to load the same complete model, rendering it unsuitable for large-scale DNN networks. On the other hand, model parallelism aims to partition the DNN model and deploy its parts to multiple processors for parallel execution [11], [12]. This method eliminates the need to load the entire model into the processor and overcomes limitations imposed by model size and device memory. Therefore, this work focuses on the literature most closely related to automatic model partitioning methods for DNNs.

Most previous research on automated model partitioning can be categorized into two broad categories as follows.

Learning-based methods: These methods treat objective functions, such as execution latency and throughput, as a black box and utilize learning-based approaches, such as reinforcement learning, to identify the appropriate partitions for a given DNN model. Mirhoseini et al. [16] leveraged reinforcement

learning-based methods to learn the model partition scheme based on the results of experts. Nevertheless, this method is much time-consuming and not applicable. Mirhoseini et al. [17] developed a reinforcement learning-based scheme that can conduct model partition and assignment simultaneously. Spotlight [18] leveraged reinforcement learning based on proximal policy optimization for model partition. In [19], the authors combined genetic algorithms and reinforcement learning together for both model partition and assignment.

Cost model-based methods: These methods construct a cost model for the model partition that closely reflects real performance. They then employ heuristic algorithms, dynamic programming, etc., to algorithmically solve the problem of finding optimal partitions of DNNs. These methods mainly focus on DNN training and aim to minimize the training throughput. For instance, GPipe [20] and PipeDream [21] introduced pipelined model-parallelism for training. HetPipe [22] proposed a hybrid data and model parallelism method for DNN training on heterogeneous GPUs. RaNNC [26] designed an automatic graph partitioning method for DNN training.

B. Optimize DNN Inference

There are extensive works [27], [28], [29] that aim to accelerate deep learning models. Here, we only mention those that aim to accelerate the DNN inference.

Model compression-based methods: These methods have two representative categories: DNN pruning and DNN quantization. DNN pruning aims to reduce the DNN model to speedup the DNN inference [27]. DNN quantization is to replace the high-precision value with low-precision value for acceleration [30].

Hardware acceleration-based methods: These methods optimize the mapping of DNN models onto specific architecture or design specific accelerators for efficient DNN execution. For example, the studies [28], [31] optimize the DNN inference on different kinds of processors, e.g., CPUs, GPUs, and FPGAs, respectively.

Task scheduling based methods: Xiang et al. [29] proposed a pipelined data parallelism scheduling for multi-DNN real-time inference. This work focuses on data parallelism scheduling to provide real-time inference services for multiple DNN models. While our work focuses on mode parallelism. Zhang et al. [32] proposed optimal task batching in intelligent edge systems for DNN inference. Jang et al. [33] optimized a real-time object detector for autonomous driving that minimizes end-to-end delay.

C. DAG Scheduling

There are two main approaches for the DAG scheduling problem: list scheduling and cluster-based scheduling [24].

List scheduling-based heuristics: The list scheduling-based heuristics [23], [34], [35], first assign a priority of each task in the DAG, sort tasks according to the task priorities, and then schedule tasks in order. There are also two types of list scheduling-based methods: static and dynamic. In the static methods, the priorities of nodes are computed once and remain unchanged throughout the scheduling process. In the dynamic methods, priorities are updated as tasks are scheduled.

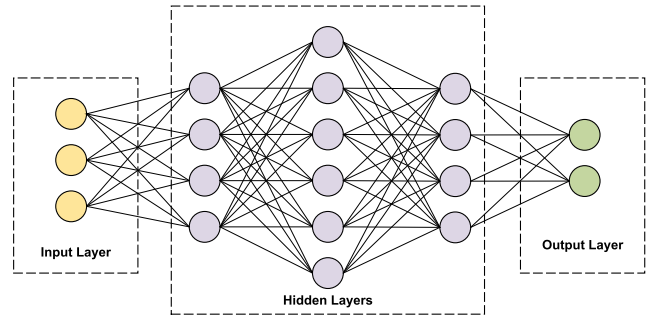


Fig. 1. Illustrative simple DNN model. It consists of five layers: an input layer for receiving input data, three hidden layers for feature extraction, and an output layer for outputting results.

Clustering based heuristics: In the cluster-based scheduling approaches [36], [37], the tasks are first partitioned into clusters. Each cluster is scheduled on the same processor. Then, the clusters are scheduled on processors. The latest state-of-the-art cluster heuristics is [37] which combines partition and scheduling together for DAG scheduling on homogeneous processors.

III. BACKGROUND AND MODELING

A. Deep Neural Networks

A DNN model can be viewed as a deeply structured data-flow model, where neurons and their connections are stacked layer by layer in a forward-pass manner [38]. Fig. 1 illustrates a simple DNN model, which consists of one input layer taking raw samples as inputs, multiple hidden layers for extracting features of data in a coarse-to-fine manner, and one output layer for outputting results.

There are two main processes for working with DNNs: the training phase and inference phase [38]. (i) During the training phase, DNNs learn model parameters from raw training data and their associated ground-truth labels. The model parameters are learned in a backward-pass manner. Note that the training process is conducted offline with the objective of enabling the trained model to perform inference tasks for practical applications. (ii) In the inference phase, trained DNNs utilize the trained model parameters to recognize unknown inputs. With the advancements in the Internet of Things (IoT) and deep learning, there is an increasing demand for utilizing trained DNNs for inference in embedded systems or edge servers [39]. Furthermore, real-time execution is a crucial factor in various applications, including autonomous driving, simultaneous interpretation, and industrial applications. Therefore, we focus on the real-time inference of DNNs in this work.

B. Problem Modeling

In this section, we primarily introduce some concepts of model partition of DNNs. Table I illustrates symbols used in this paper.

1) *Operation DAG of DNNs:* Popular computing frameworks, such as TensorFlow, PyTorch, and MxNet, perform the design and implementation of a DNN by building and executing

TABLE I
NOTATIONS

Notations	Meaning
$G(V, E)$	The DNN graph.
V, E	The node set and edge set of G .
m, n	The total number of edges/nodes in G .
$v_i \in V$	The node v_i denotes an operation/task
$e_{(i,j)} \in E$	The edge between node i and j .
P	The processor set.
q	The total number of processors.
$p_i \in P$	The i -th processor.
D	The $n \times n$ matrix of communication data.
$D_{(i,j)}$	The amount of data from task v_i to task v_j .
W	The $n \times q$ computation cost matrix.
T	The data transfer rates matrix between the heterogeneous processors with $q \times q$ size.
S	The communication startup cost vector of processors with length q .
$c_{(v_i, v_j)}$	The communication cost of edge $e_{(v_i, v_j)}$.

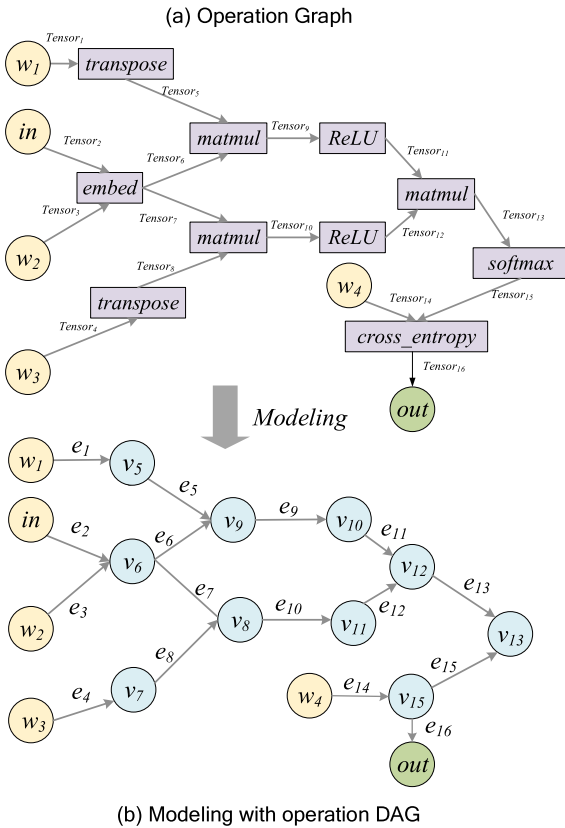


Fig. 2. (a) A simple DNN structure with its operation graph. (b) Modeling the graph with operation DAG.

operation computing graphs. Fig. 2(a) illustrates an example of an operation graph for a DNN. The DNN model takes raw data samples and parameters as input and performs the relevant operations to generate the corresponding results. In Fig. 2(a), yellow circles represent input data (raw data or parameters), blue rectangles represent operations, and green circles represent output data. The directed edges represent tensors that flow among nodes, and each node is associated with the tensor data required

for execution. Operation includes various mathematics, arrays, matrices, states, and neural network construction, including *Sub*, *Add*, *Mul*, *MatMul*, *Sigmoid*, *SoftMax*, *ReLU*, *Convolution2D*, etc.

Based on the above description, the operation graph of DNN can be further modeled as an operation DAG, as shown in Fig. 2(b). The operation DAG can be denoted as $G = (V, E)$, where V and E are the node/edge set. Each node $v_i \in V$ denotes an operation, which is also treated as a task (*node* and *task* terms are interchangeably used in this work). $n = |V|$ denotes the total number of tasks. Note that, we also model the input data (including input data samples and input parameters) as nodes, which are depicted as green circles. In contrast, the nodes of operations are depicted as blue circles. The directed edge $e_{(i,j)} \in E$ encodes dependency or precedence constraints among tasks, i.e., $e_{(i,j)}$ means that the operation v_i relies on the result of v_j .

In addition, We employ $Pred[v_i] = v_j | (v_j, v_i) \in E$ to represent the (immediate) predecessors of the node v_i , and $Succ[v_i] = v_j | (v_i, v_j) \in E$ to denote the (immediate) successors of v_i in G . Nodes without any predecessors are termed *source* nodes, and those without successors are termed *target* nodes. D is a $n \times n$ communication data matrix, where $D_{(v_i, v_j)}$ is the amount of data that needs to be transferred from task v_i to task v_j .

2) *Execution on Heterogeneous Multi-Core System*: In this work, we consider developing a heterogeneous multicore system, where the main memory is shared among all CPUs. The system is equipped with one or more kinds of CPU cores and accelerators (GPU, FPGA, etc.), i.e., $P = \{p_1, p_2, \dots, p_i, \dots, p_q\}$ where p_i is either a CPU core or an accelerator, q is the total number of cores.

To represent the execution of an operation DAG of a DNN on a heterogeneous multi-core system. Some definitions are given as follows. $W \in \mathbb{R}^{n \times q}$ is a computation cost matrix, where each $W_{(i,j)}$ denotes the estimated execution time to complete task v_i on a processor. For the input data nodes that do not require computation, the weight is set as 0. The communication cost is paid only if v_i and v_j are placed on different devices p_k and p_l . The data transfer ratio among the heterogeneous processors is stored in matrix $T \in \mathbb{R}^{q \times q}$. The communication startup costs of processors are denoted by a vector $S \in \mathbb{R}^{1 \times q}$. Then, the communication cost of edge $c_{(v_i, v_j)}$, i.e., the data transfer from task v_i (scheduled on p_k) to task v_j (scheduled on p_l), is

$$c_{(v_i, v_j)} = S_k + \frac{D_{(v_i, v_j)}}{T_{(k, l)}}. \quad (1)$$

3) *Problem Formulation*: To achieve real-time inference for large-scale DNNs, we aim to schedule each node of the operation DAG to a processing core in a heterogeneous multi-core system. The primary goal is to minimize the *makespan* of the DNN execution while ensuring that the capacity of each processing core is not exceeded.

During the scheduling process, operations that are scheduled on the same processor are grouped together as a partition. Meanwhile, since the raw data and parameters are modeled as nodes in the operation DAG, the model parameters can also be partitioned according to the partition results. This ensures that

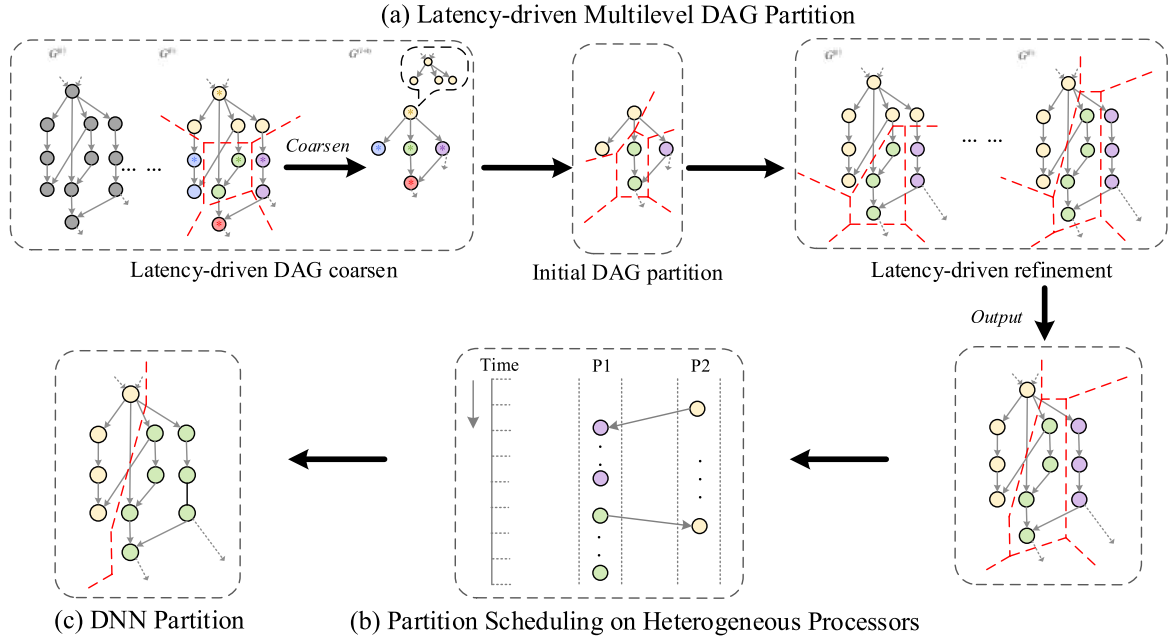


Fig. 3. Workflow of the proposed SAMP framework. SAMP comprises three sub-procedures: (a) Latency-driven DAG Partition, (b) Partition Scheduling on Heterogeneous Processor, (c) DNN Partition.

the data required for each partition is appropriately distributed across the processing cores.

IV. PROPOSED METHODOLOGY

A. Methodology Overview

The proposed SAMP aims at automated model partitioning for real-time inference of large-scale DNNs on heterogeneous processors. There are two primary objectives of our proposed SAMP: minimizing the execution time of partitioned DNNs on heterogeneous processors for the large-scale DNN models, and ensuring low time complexity to enable efficient partitioning. Fig. 3 illustrates the overall framework of the solution. It comprises three sub-procedures which are shown as follows.

Latency-driven Multilevel DAG Partition (LMDP): In this phase, we consider utilizing a recent DAG partitioner: DAGP [40]. Nevertheless, the original DAGP does not take the execution time of the DAG and the memory constraint into consideration. To this end, we propose a novel latency-driven multilevel DAG partition algorithm during this phase, termed LMDP, specifically designed for partitioning large-scale DAGs. It includes a latency analysis step and a multilevel acyclic graph partitioning step. The multilevel acyclic graph partitioning is conducted for the given operation DAG of a DNN using latency and topology information obtained in the latency analysis. Note that the resulting partitions generated by LMDP are acyclic. Thus, each partition can be completely scheduled once the input tasks are ready for execution.

Partition Scheduling on Heterogeneous Processors (PSHP): Based on the results of LMDP, we combine a partition part as a super node, update edges for the super nodes, and update attributes of the partitioned DAG for scheduling the partitioned DAG on heterogeneous processors with memory constraints. In this phase, we propose a novel PSHP algorithm to minimize the

execution time of partition DAG. Since the operation DAG of DNN is reduced after the first DAG partition phase, the search space for the second scheduling phase is reduced.

DNN Model Partition: Based on the scheduling results, operations scheduled on the same processor are grouped together as a partition. Simultaneously, the model parameters are partitioned based on the final DAG partition results.

Remarks: Our proposed SAMP is similar to cluster-based scheduling methods in that they both divide tasks into clusters. However, SAMP differs significantly from cluster-based scheduling methods in the following aspects:

(1) Unlike previous cluster-based scheduling methods, partitions in SAMP satisfy four conditions: a) Minimization of weighted edge cuts. Data communication is an important factor in heterogeneous scheduling, the size of data communicated between the final DNN model partition should be small to reduce the communication overhead. Weighted edge cuts denote the total data communications among partitions. b) Balanced weight of the partition parts, i.e., the total vertex weight of each part does not exceed $(1 + \epsilon) \frac{\sum_{v_i \in V} w_i}{K}$; Load balance is also an essential factor that impacts the performance of parallel computation. c) The partition is acyclic, ensuring that partitions can be scheduled on heterogeneous processors. d) Minimize the critical path of the DAG. The critical path of the DAG is in essence the lower bound of the schedule lengths generated by the task scheduling algorithms. Minimizing the critical path of the DAG during the partition would improve the performance of the next scheduling step.

(2) SAMP employs the list scheduling method for task scheduling. Therefore, it can be viewed as a hybrid combination of list-scheduling and cluster-scheduling methods.

(3) In contrast to the cluster-based scheduling method that focuses on scheduling DAG tasks, our work aims at partitioning DNN models for real-time DNN inference.

B. Latency-Driven Multilevel DAG Partition (LMDP)

In this section, we introduce the proposed LMDP algorithm as shown in Fig. 3(a). In LMDP, we take a DAG with nodes and edge weights, the partition number K , and an allowed imbalanced parameter ϵ as inputs. The output of LMDP is a partition G into K parts, i.e., $G = \{g_1, g_2, \dots, g_K\}$, where for any two parts g_i and g_j , $g_i \cup g_j = \emptyset$ and $\sum_{i=1}^K g_i = G$. There are four main phases in LMDP: latency analysis, latency-driven DAG coarsening, initial DAG partition, and latency-driven refinement, as shown in Fig. 3(a). Details of the proposed LMDP are presented as follows.

Latency Analysis: First, we calculate the average processing time $\overline{w}_{v_i}$ of each DNN node v_i on the heterogeneous processors and the average transfer time $\overline{c}_{(v_i, v_j)}$ of each DNN edge $e_{(i, j)}$, i.e.,

$$\begin{aligned} \overline{w}_{v_i} &= \frac{\sum_{k=0}^q W_{(i, k)}}{q}, \\ \overline{S} &= \frac{\sum_{k=0}^q S_k}{q}, \\ \overline{c}_{(v_i, v_j)} &= \overline{S} + \frac{D_{(i, j)}}{\overline{T}}, \end{aligned} \quad (2)$$

where q is the total number of processors, k is the index of the processor, $\overline{S}$ denotes the average communication startup time, and $\overline{T}$ denotes the average transfer rate among the processors in the domain.

Then, the DNN DAG is traversed forward and backward using the average transfer time and average processing time to acquire the latency ranking of each node u , i.e.,

$$\begin{aligned} RankD_{(v_i)} &= \max_{v_j \in Pred[v_i]} (RankD_{(v_j)} + \overline{w}_{v_i} + \overline{c}_{(v_j, v_i)}), \\ RankU_{(v_i)} &= \max_{v_j \in Succ[v_i]} (RankU_{(v_j)} + \overline{w}_{v_j} + \overline{c}_{(v_i, v_j)}). \end{aligned} \quad (3)$$

When the node v_i is the *source* node, the value of $RankD_{(v_i)}$ is set as $\overline{w}_{v_i}$. When the node v_j is the *target* node, the value of $RankU_{(v_j)}$ is set as 0.

Third, according to $RankD$ and $RankU$, we can get the information of the longest path, i.e., the path with the most delay cost, of any directed edge in the DAG. Formally, the equation is,

$$\begin{aligned} path(e) &= RankD(begin(e)) + w(e) + \overline{w}_{end(e)} \\ &\quad + RankU(end(e)), \end{aligned} \quad (4)$$

where the $begin(e)$ denotes the begin node of a directed edge e , while $end(e)$ denotes the end node of a directed edge e .

Finally, to facilitate the computation and update under multi-level partition, we define a new weight value $W(e)$ for the DNN graph edge as follows,

$$\begin{aligned} path_{min} &= \min_{e \in E} (path(e)), \\ W(e) &= path(e) - path_{min}. \end{aligned} \quad (5)$$

where $path_{min}$ is the shortest latency path for the entire DNN graph, and $W(e)$ is the difference among the longest latency path for each edge and $path_{min}$. The value of $W(e)$ reflects the

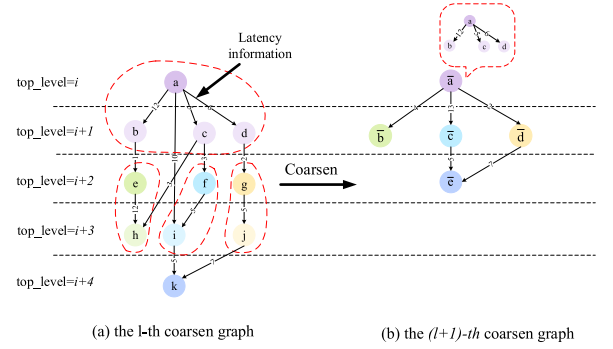


Fig. 4. Illustrative example of acyclic partitioning.

effect of this edge on the worst execution time and the overall execution time. That is, the smaller $W(e)$ of the edge e , the less impact it has on the overall execution time, and the more suitable for cutting.

Latency-driven DAG Coarsening: In this phase, we merge nodes in the DAG into super nodes to coarsen the graph, as shown in Fig. 4. The DAG of the l -th layer for coarsening is denoted as $G^{(l)} = (V^{(l)}, E^{(l)})$. First, the topological order of each node in the $G^{(l)}$ is computed based on the previous phase and utilizes the calculated latency information $W(e)$ as the edge weights. Then, all neighbors $Nei(v_i^{(l)})$ of node $v_i^{(l)}$ are computed whether they satisfy our defined matching conditions. If the neighbor nodes are matched with $v_i^{(l)}$, they are marked as matching nodes $M_i^{(l)}$ for the following merging. If the matching conditions are not satisfied, they are not marked.

The matching conditions can be defined as:

Definition IV.1: Suppose $G^{(l)} = (V^{(l)}, E^{(l)})$ is a DAG in the l -th layer. $M^{(l)} = \{M_1^{(l)}, \dots, M_k^{(l)}\}$ is denoted as a set of matching super nodes. A matching super node is a set of disjoint subsets of nodes that satisfy the following conditions:

- For any nodes u, v in a matching super node, $|top_level(u) - top_level(v)| \leq 1$.
- For matching super nodes M_i and M_j , and every $u \in M_i^{(l)}$, and $v \in M_j^{(l)}$, either $(u, v) \notin E$ or $|top_level(u) - top_level(v)| \neq 1$.

The weight of the matching super node is equal to the sum of weights of all nodes in this matching super node. The node weight $w(v)$ and edge weight $w(e)$ in the $(l+1)$ -th coarsen layer is formally defined as follows.

$$\begin{aligned} w(e_{i, j}^{(l+1)}) &= \sum_{u \in M_i^{(l)}, v \in M_j^{(l)}} w(e_{u, v}^{(l)}), \\ w(v_i^{(l+1)}) &= \sum_{t \in M_i^{(l)}} w(v_t^{(l)}). \end{aligned} \quad (6)$$

Different from the direct partition of the raw DAG, this method can gain partition efficiency and quality: (1) Coarsening the DAG helps reduce the size and complexity of the graph. By merging nodes and edges, the number of vertices and edges is reduced, thereby reducing the computational effort and memory requirements of the partitioning process. (2) Coarsening enables

better partition quality. The coarser graph retains the structural characteristics of the original graph, i.e., the connectivity patterns and dependencies among nodes, which are essential for maintaining the integrity of the partitioned subgraphs. As a result, partitioning the coarser graph is more likely to result in balanced and cohesive subgraphs, leading to better load balancing and improved performance.

Initial DAG Partition: During delayed path coarsening, we first coarse the nodes on the longer path. Follow this line, we also first partition the nodes of the longer path in the initial DAG partition. The recursive bi-partition is utilized based on the latency-path in the initial DAG partition. First, we choose a node with the minimum value of $RankD$, and then set it in a specific partition (e.g., part 0). After that, the depth-first extension approach is applied to place other traversal nodes in the same partition until the equilibrium condition is satisfied. According to these two mechanisms, nodes on the same path can be selected into the same partition, thus avoiding increasing the execution latency of the operation DAG.

Latency-driven Refinement: Since the initial partition is performed for the coarsened DNN graph, it requires to be mapped back to the raw DNN graph. We use the latency-driven refinement phase to gradually map the initial partition to the original graph. The boundary FM algorithm [41] is utilized in this phase. It computes the gain of each node at the partition boundary based on the optimization objective. Then, it constructs the gain queue according to all the gains obtained. We modify the directed FM according to the latency information. Then the gain for moving a boundary node for v_i is

$$gain(v_i) = \sum_{v_j \in Succ[v_i]} W(v_i, v_j) - \sum_{v_j \in Pred[v_i]} W(v_j, v_i). \quad (7)$$

Memory Constraints: We take the memory constraints into consideration during the coarsening, initial partition, and refinement phases. For the operation execution of DNNs, the memory consumption includes both intermediate tensors and input parameters. Actually, intermediate tensors can be released if they are not required for subsequent tasks. Therefore, for a specific partition, the memory consumption of this partition contains two parts. One is the memory consumption of input parameters in this partition. The other part is the memory consumption of the task which requires the largest sizes of intermediate tensors for execution. We add these two parts together as the memory consumption of this partition. We then define a maximum memory size M_p to dynamically control the memory size for each partition in this phase. During the coarsening, initial partition, and refinement phases, if a node movement can make the memory consumption of a partition exceed the predefined M_p , this node movement will be prohibited.

C. Partition Scheduling on Heterogeneous Processors (PSHP)

PSHP algorithm aims to schedule the partitioned DAG on q heterogeneous processors with the objective of minimizing the execution time of the partitioned DAG. It is motivated by the HEFT and CPop [23] algorithms, which have achieved excellent

performance on scheduling DAG of tasks on heterogeneous processors with relatively low complexity. There are two primary steps in PSHP: (1) Partition prioritizing phase that calculates the priorities of all partitions in the partitioned DAG, (2) Processor selection phase that selects the partitions based on the rank of priorities and schedules each selected task on its “best” processor to minimize the task completion time. The pseudocode of PSHP is illustrated in Algorithm 1.

Calculating Priorities of the Partitioned DAG: After the LMDP phase, the original operation DAG of the DNN is partitioned into K acyclic subgraphs. We then abstract each part as a super node, and abstract super edges among super nodes. The super node set is denoted as SV , while the super set is denoted as SE . A super node contains several original nodes in the original DAG. The $K \times q$ computation cost matrix SW for the super nodes is calculated as follows:

$$sw_{(sv_i, k)} = \sum_{sv_k \in SV_i} w_{(sv_k, sv_j)}, \quad (8)$$

where i is the index of the super node, and k is the index of the processor. For each processor, we sum the weights of all the original nodes in a super node up to denote the weight of the super node.

The $K \times K$ matrix of communication data SD for the super nodes is calculated as follows:

$$sd_{(sv_i, sv_j)} = \sum_{e \in se_{(sv_i, sv_j)}} sd_e, \quad (9)$$

where $se_{(v_i, v_j)}$ is the super edge between super node sv_i and super node sv_j .

Then we calculate the average processing time $\overline{sw}_{sv_i}$ of each super node $sv_i \in SV$ on the heterogeneous processors and the average transfer time $\overline{sc}_{(sv_i, sv_j)}$ of each super edge $se_{(sv_i, sv_j)}$, i.e.,

$$\begin{aligned} \overline{sw}_{sv_i} &= \frac{\sum_{k=1}^q W_{(sv_i, k)}}{q}, \\ \overline{sc}_{(sv_i, sv_j)} &= \overline{S} + \frac{D_{(sv_i, sv_j)}}{\overline{T}}. \end{aligned} \quad (10)$$

For each super node sv_i , we can perform the forward and backward traversal like the latency analysis to get the scheduling priority, i.e.,

$$\begin{aligned} Priority(sv_i) &= \max_{sv_j \in Pred[sv_i]} (Priority(sv_j) + \overline{sw}_{sv_i} \\ &\quad + \overline{sc}_{(sv_j, sv_i)}), \end{aligned} \quad (11)$$

The larger the value of $Priority(sv_i)$, the lower the priority of part sv_i . When there are parts with the same $Priority$, we randomly choose their orders.

After the scheduling priority is determined, under the constraints of memory, we assign each part, according to the order of the priority of the part, to a processor to minimize the *makespan*. Before giving the formula of *makespan*, we need to introduce two values, i.e., the earliest execution start time and the earliest execution finish time of each part N_i in a processor p_j , i.e., $EEST(sv_i, p_j)$ and $EEFT(sv_i, p_j)$, respectively. The

Algorithm 1: Partition Scheduling on Heterogeneous Processors.

```

1: Compute the computation cost for super nodes via (8)
2: Compute the computation cost for edges between super
   nodes via (9)
3: Compute the Priority of each super node via (11)
4: Sort super nodes in a list by nonincreasing order based
   on Priority value
5: while there are unscheduled nodes in the list do
6:   Select the first super node  $sv_i$  from the list
7:   for each processor  $p_j$  in the processor set do
8:     Compute  $ETF(sv_i, p_j)$  via (12)
9:   end for
10:  Assign all the original nodes in the super node  $sv_i$  to
     minimize  $EFT$  of super node  $sv_i$  with memory
     constraints.
11: end while
12: return

```

$EEST$ and $EEFT$ are calculated recursively, starting from the entry part as follows:

$$\begin{aligned}
EST(sv_{entry}, p_j) &= 0, \\
EFT(sv_i, p_j) &= w_{(sv_i, p_j)} + EEST(sv_i, p_j), \\
EST(sv_i, p_j) &= \max\{avail[j], \\
&\quad \max_{sv_m \in Pred[sv_i]} (AFT(sv_m) + sc_{(m, i)})\} \quad (12)
\end{aligned}$$

where $Pred[sv_i]$ denotes the set of immediate predecessor of super node sv_i , $avail[j]$ denotes the earliest time at which processor p_j is ready to execute a task. The inner max block in the equation returns the *readytime*, i.e., the time when all the required data has arrived at processor p_j .

The formula for calculating *makespan* is

$$makespan = \max\{AFT(sv_{exit})\}, \quad (13)$$

where $AFT(sv_{exit})$ is the actual completion time of implement the exit node sv_{exit} . In this manner, we can map tasks onto processors, which allows us to satisfy the task priority requirement and obtain the minimum completion.

Heterogeneous Processor Selection for Partitions: Similar with the HEFT algorithm, the earliest available time for processor p_i to perform a partition is the time when p_i completes the previously allocated partition. Furthermore, scheduling on this idle time slot should preserve the priority constraints for super nodes. If a processor is selected for executing a partition (a super node), all the original nodes in this super node will be assigned to this processor. Similarly, we define a maximum memory size M_s to dynamically control the maximum memory sizes for all the super nodes on a processor in this phase. For all the original nodes, we add memory sizes of input parameters and the node that requires the largest intermediate tensors as the maximum memory consumption of this partition. During the processor selection phase, if a node movement can make the maximum memory size of a partition exceed the predefined M_s , this node movement will be prohibited.

D. Computational Complexity Analysis

The computational complexity of our SAMP is analyzed as follows. Suppose the number of partitions generated by the first phase is K , the number of super edges is r , the number of processors is p , the number of nodes is n and the total number of edges is m . The LMDP algorithm has the computational complexity of $O(n + m)$, while the PSHP algorithm is $O(r \times p)$. r is much smaller than m . We can observe that the computational complexity of SAMP has a linear relationship with the total number of edges.

V. EXPERIMENTAL RESULTS

In this section, we first introduce the experiment setup, and then evaluate the effectiveness of the algorithm from two aspects: the overall execution latency of the model and the running time of the partition algorithm.

A. Experiment Setup

1) **Hardware Platform:** For a fair comparison, we leverage the same settings as the IP [15]. The environment includes a CPU and 6 accelerators. The utilized CPU has 8 cores. The DRAM size of each accelerator is assumed to be 8 GB. The CPU and accelerators are connected via PCIe 3.0. This experimental setup can be scaled to any heterogeneous processing topology. All experiments are performed on a server with an Intel Xeon E5-2673 v4 CPU and 64 GB RAM. The server is installed with Ubuntu 18.04.

2) Compared Baselines:

- **IP [15]:** The Integer Programming (IP) method is the latest state-of-art approach on model partition, which is utilized to minimize the model execution latency. The source code can be found in.¹
- **DP [15]:** It is the latest state-of-art approach on model partition using Dynamic Programming (DP) method. We utilize latency minimization as the target of IP algorithm for fair comparison.
- **Metis [42]:** It is the most popular graph partition algorithm, which aims to partition large-scale graphs into balanced subgraphs with a small amount of cut edges.
- **DAGP [41] and memDHGP [43]:** They are two latest state-of-art directed acyclic graph partitioning algorithms.
- **HEFT [23] and RFID [44]:** They are two popular list-based heterogeneous DAG scheduling methods.
- **SCDAG [37]:** It is a popular cluster-based DAG scheduling method.

3) **DNN Models:** We conduct experiments on broadly used deep learning workloads to evaluate the efficiency and scalability of our SAMP, ranging from small models, and medium models to large models. We also utilize an automatic model design approach termed ENAS [45] for generating diverse neural network architectures. We export the operator graphs of BERT [46] and ResNet [47] via the ONNX Runtime library.² It allows us to derive the operator graph topology for DNN models by feeding

¹<https://github.com/msr-fiddle/dnn-partitioning>

²<https://github.com/onnx/onnx>

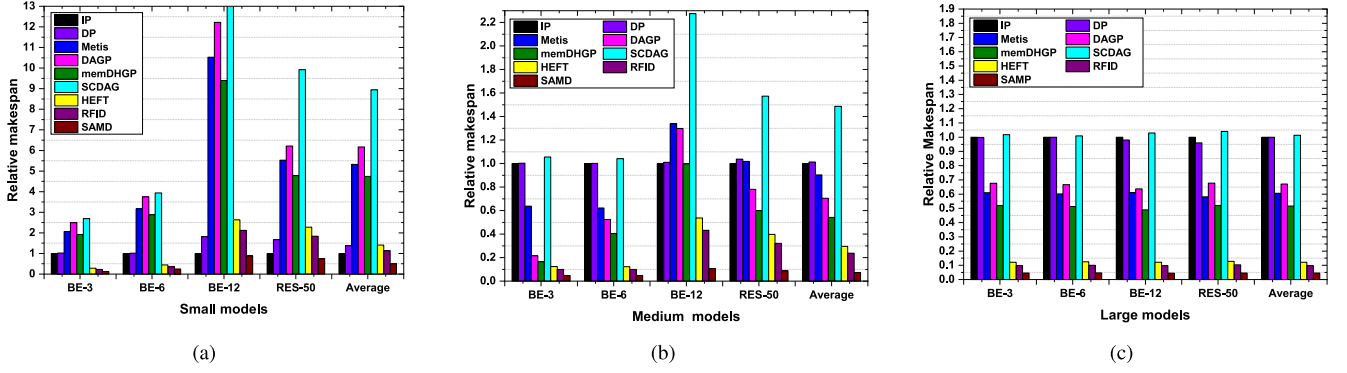


Fig. 5. Performance comparison of the execution latency (*makespan*). (a) Small models. (b) Medium models. (c) Large models.

TABLE II
DETAIL INFORMATION OF EVALUATED DATASETS

Type	Graphs	Nodes	Edges	Density
Small	BE-3	235	250	1.06
	BE-6	418	448	1.07
	BE-12	783	844	1.08
	RES-50	604	619	1.02
Medium	BE-96	5908	6388	1.08
	BE-128	7860	8500	1.08
	RES-101	1182	1214	1.03
	RES-152	1760	1809	1.03
Large	BE-256	15668	16948	1.08
	BE-512	31284	33844	1.08
	NAS-1	42498	48022	1.13
	NAS-2	56781	67569	1.19

their forward pass as input and attaching the corresponding backward pass to produce the output in ONNX format.

The details of DAG for DNNs are presented in Table II. For simplicity, we use BE to denote BERT and RES to denote ResNet. BE-3 means the BERT model with 3 layers, while RES-50 means the ResNet model with 50 layers, NAS-1 and NAS-2 are the networks generated by [45].

B. Performance of Execution Latency

The performance comparison of execution latency obtained by different algorithms is shown in Fig. 5. In order to more intuitively describe the performance of different algorithms, we show the relative *makespan* of different algorithms compared with the IP algorithm, which is the latest state-of-the-art work for model partition. We set the number of partitions generated by LMDP k as 60 for both small and medium DNN models, and 100 for large DNN models. Detailed results on different numbers of partitions are analyzed in Section V-D.

Fig. 5(a) shows the performance comparison of small models. The average relative *makespan* of DP, Metis, DAGP, memDHGP, SCDAG, HEFT, RFID, and SAMP is 1.38, 5.32, 6.16, 4.75, 8.94, 1.40, 1.14, and 0.49 respectively. The algorithm which performs better obtains the lower value of the relative *makespan*.

Observations can be obtained as follows. (i) HEFT and RFID algorithms perform better than IP and DP algorithms for BE-3 and BE-6 models, however, they perform worse than IP and DP algorithms for BE-12 and RES-50 models. It shows that the heterogeneous scheduling algorithms (i.e., HEFT and RFID) have effects on DNN model partition problem. However, it has room to improve. (ii) The algorithms Metis, DAGP, memDHGP, and SCDAG perform much worse than IP and DP algorithms. It shows that the graph or DAG partitioning algorithms do not perform well on small models. (iii) HEFT and RFID algorithms perform better than Metis, DAGP, memDHGP, and SCDAG algorithms for all the small models, which demonstrates that the scheduling methods perform better than the graph or DAG partitioning algorithms on small models.

Our SAMP performs better than all the algorithms, which demonstrates the effectiveness of SAMP for partitioning small models. Specifically, SAMP obtains almost $2\times$ improvement compared with the IP algorithm, and obtains $3.5\times$ improvement compared with the HEFT algorithm on small models on average.

Performance of Medium Models: Fig. 5(b) shows the performance comparison of medium models. The average relative *makespan* of DP, Metis, DAGP, memDHGP, SCDAG, HEFT, RFID, and SAMP is 1.01, 0.90, 0.70, 0.54192, 1.48, 0.29, 0.24, and 0.07 respectively.

We can observe that HEFT and RFID perform better than IP and DP algorithms. Algorithms Metis, DAGP, memDHGP, and SCDAG also perform better than the IP and DP algorithms. It shows that the performance of IP and DP algorithms is decreased when the scale of models is increased from small to large. Our SAMP performs much better than all the algorithms on medium models. Specifically, SAMP obtains almost $14.2\times$ improvement compared with the IP algorithm, and obtains $4.1\times$ improvement compared with the HEFT algorithm, $10\times$ improvement compared with DAGP and $21.1\times$ improvement compared with SCDAG.

Performance of Large Models: Fig. 5(c) shows the performance comparison of large models. The average relative *makespan* of DP, Metis, DAGP, memDHGP, SCDAG, HEFT, RFID, and SAMP is 0.99, 0.60, 0.67, 0.52, 1.01, 0.12, 0.98, and 0.04 respectively. Our SAMP performs better than all the algorithms on large models. Specifically, SAMP obtains almost $25\times$ improvement compared with the IP algorithm, and obtains

TABLE III
AVERAGE RUNNING TIME OF DIFFERENT ALGORITHMS

Methods	Small				Medium				Large			
	BE-3	BE-6	BE-12	RES-50	BE-96	BE-128	RES-101	RES-152	BE-256	BE-512	NAS-1	NAS-2
IP	109.79s	>10h	>10h	>10h	>10h	>10h	>10h	>10h	>10h	>10h	>10h	>10h
DP	5.12s	61.12s	62.34s	30.81s	20.43s	50.46s	5.23s	5.14s	20.45s	20.51s	30.43s	40.87s
Metis	0.004s	0.004s	0.004s	0.008s	0.01s	0.01s	0.01s	0.01s	0.02s	0.03s	0.03s	0.04s
DAGP	0.01s	0.01s	0.02s	0.02s	0.09s	0.11s	0.03s	0.03s	0.22s	0.47s	0.59s	0.79s
SCDAG	0.03s	0.03s	0.05s	0.06s	0.60s	0.53s	0.14s	0.23s	3.03s	6.19s	8.03s	9.19s
HEFT	0.005s	0.07s	0.09s	0.09s	3.14s	5.61s	0.17s	0.38s	23.54s	213.13s	387.43	598.98s
RFID	0.009s	0.12s	0.017s	0.017s	8.07s	11.88s	0.39s	0.72s	27.12s	252.74s	400.09	712.62s
SAMP	0.02s	0.03s	0.04s	0.04s	0.21s	0.29s	0.06s	0.08s	0.69s	1.39s	2.49s	3.89s

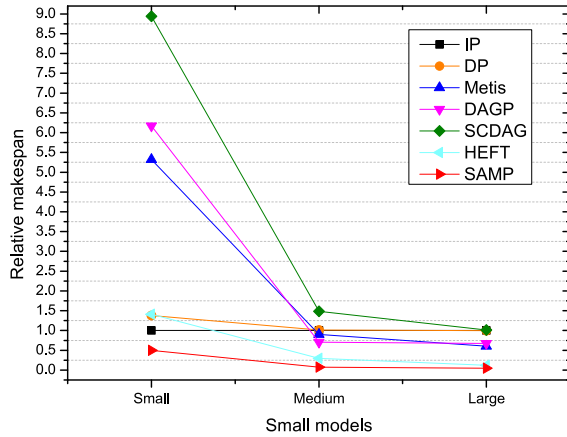


Fig. 6. Average relative *makespan* on small, medium, and large models.

$3\times$ improvement compared with the HEFT algorithm on large models, $16.7\times$ improvement compared with DAGP and $25.2\times$ improvement compared with SCDAG. It demonstrates that our SAMP can obtain the shortest execution time compared with other baselines for model parallelism.

Summary: Fig. 6 shows the average relative *makespan* on small, medium, and large models for different algorithms. Our SAMP can obtain the shortest execution time compared with other baselines for both small, large, and enormous models. Most importantly, we have found that SAMP achieves the best comparative performance for enormous models, which demonstrates the scalability of SAMP.

C. Results of Running Time

Table III demonstrates the average running time of different algorithms. The DAG partitioning methods (including Metis and DAGP), which only consider the minimization of the number of cutting edges during partitioning, adopt the multi-level partitioning approach to obtain the shortest running time. However, they do not guarantee the minimum overall execution time of the model. The running time of HEFT algorithm for small models is similar to that of Metis and DAGP. However, the running time of HEFT increases significantly as the model size increases.

We can observe that the running time of SAMP is much shorter than that of HEFT, and results can be obtained within a few seconds. Although our SAMP has a longer running time than the DAG partitioning methods, it optimizes the overall execution

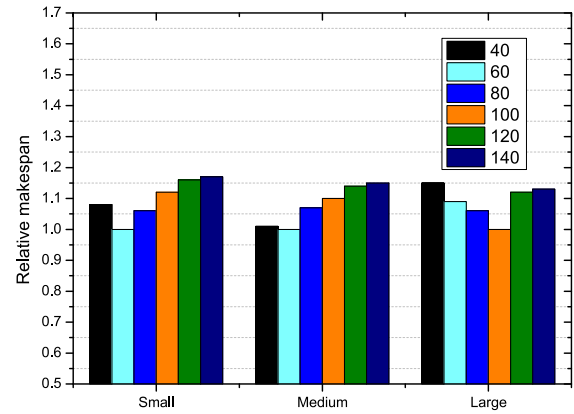


Fig. 7. Average relative *makespan* with different partitions.

time by considering dynamic scheduling and resource allocation, improving the overall execution efficiency of the model. Thus, our algorithm achieves the optimal tradeoff between the overall execution delay of the model and the running time of the partitioning algorithm.

D. Results of Different Partitions Generated by LMDP

In LMDP, the number of partitions is a key parameter that affects the overall performance. Specifically, too few partitions would cause some processors to overload and limit parallelism. Conversely, too many partitions result in a large amount of data exchange and increase the communication overhead. Therefore, it is critical to choose an appropriate number of partitions. In Fig. 7, we study the effect of different partitions on the average relative performance of LMDP in small, medium, and large models. We set the smallest *makespan* to 1. The results demonstrate that the optimal partitions are different for models of different sizes. For small, medium, and large models, LMDP achieves the best performance on 60, 60, and 100 partitions, respectively.

VI. CONCLUSION

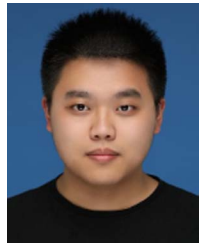
Deep neural networks (DNNs) have achieved remarkable success in various domains, such as image recognition and natural language processing. Scaling up DNNs is an effective method to improve model quality, making the DNN models continuously grow. Moreover, to speedup the execution efficiency of large and complex models, the devices are becoming increasingly

heterogeneous. In this paper, we propose a scalable automated model partition method, SAMP, for real-time inference of large-scale DNNs on heterogeneous processors by exploring ideas in heterogeneous DAG scheduling methods. SAMP is a hybrid DAG partition and DAG scheduling method, aiming to minimize the execution time of DNN models by model parallelism while considering memory constraints and low computation complexity. We demonstrate the applicability, efficiency, and scalability of our approaches using both small and large-scale DNN models. Future work could focus on exploring heterogeneous scheduling methods to optimize the training throughput of DNNs.

REFERENCES

- [1] N. Han, J. Chen, G. Xiao, H. Zhang, Y. Zeng, and H. Chen, "Fine-grained cross-modal alignment network for text-video retrieval," in *Proc. 29th ACM Int. Conf. Multimedia*, 2021, pp. 3826–3834.
- [2] X. Zou, K. Li, and C. Chen, "Multilevel attention based u-shape graph neural network for point clouds learning," *IEEE Trans. Ind. Inform.*, vol. 18, no. 1, pp. 448–456, Jan. 2022.
- [3] Y. Li, C. Chen, M. Duan, Z. Zeng, and K. Li, "Attention-aware encoder-decoder neural networks for heterogeneous graphs of things," *IEEE Trans. Ind. Inform.*, vol. 17, no. 4, pp. 2890–2898, Apr. 2021.
- [4] T. B. Brown et al., "Language models are few-shot learners," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [5] C. Chen, K. Li, X. Zou, and Y. Li, "DyGNN: Algorithm and architecture support of dynamic pruning for graph neural networks," in *Proc. 58th IEEE/ACM Des. Automat. Conf.*, 2021, pp. 1201–1206.
- [6] C. Chen, K. Li, Y. Li, and X. Zou, "ReGNN: A redundancy-eliminated graph neural networks accelerator," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit.*, 2022, pp. 429–443.
- [7] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–14.
- [8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2022 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.
- [9] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, pp. 1–67, 2020.
- [10] D. Lepikhin et al., "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–23.
- [11] S. Moreno-Alvarez, J. M. Haut, M. E. Paoletti, and J. A. Rico-Gallego, "Heterogeneous model parallelism for deep neural networks," *Neurocomputing*, vol. 441, pp. 1–12, 2021.
- [12] F. Xu, J. Feng, Z. Zhang, L. Ning, and Y. Jia, "EdgeMesh: A hybrid distributed training mechanism for heterogeneous edge devices," *Trans. Emerg. Telecommun. Technol.*, vol. 34, no. 3, 2023, Art. no. e4706.
- [13] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks," in *Proc. Mach. Learn. Syst.*, 2019, pp. 1–13.
- [14] S. Ö. Arik et al., "Deep voice: Real-time neural text-to-speech," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 195–204.
- [15] J. M. Tarnawski, A. Phanishayee, N. Devanur, D. Mahajan, and F. Nina Paravecino, "Efficient algorithms for device placement of DNN graph operators," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 15451–15463.
- [16] A. Mirhoseini et al., "Device placement optimization with reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2430–2439.
- [17] A. Mirhoseini, A. Goldie, H. Pham, B. Steiner, Q. V. Le, and J. Dean, "A hierarchical model for device placement," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–11.
- [18] Y. Gao, L. Chen, and B. Li, "Spotlight: Optimizing device placement for training deep neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1676–1684.
- [19] S. Bojja Venkatakrishnan et al., "Learning generalizable device placement algorithms for distributed machine learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 3981–3991.
- [20] Y. Huang et al., "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 103–112.
- [21] Z. Jia, M. Zaharia, and A. Aiken, "Beyond data and model parallelism for deep neural networks," in *Proc. Mach. Learn. Syst.*, 2019, pp. 1–13.
- [22] J. H. Park et al., "HetPipe: Enabling large DNN training on (whimpy) heterogeneous GPU clusters through integration of pipelined model parallelism and data parallelism," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 307–321.
- [23] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
- [24] H. Wang and O. Sinnen, "List-scheduling versus cluster-scheduling," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 8, pp. 1736–1749, Aug. 2018.
- [25] L. Qin, Y. Gong, T. Tang, Y. Wang, and J. Jin, "Training deep nets with progressive batch normalization on multi-GPUs," *Int. J. Parallel Program.*, vol. 47, no. 3, pp. 373–387, 2019.
- [26] M. Tanaka, K. Taura, T. Hanawa, and K. Torisawa, "Automatic graph partitioning for very large-scale deep learning," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2021, pp. 1004–1013.
- [27] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–17.
- [28] Y. Liu, Y. Wang, R. Yu, M. Li, V. Sharma, and Y. Wang, "Optimizing CNN model inference on CPUs," in *Proc. USENIX Annu. Tech. Conf.*, 2019, pp. 1025–1040.
- [29] Y. Xiang and H. Kim, "Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference," in *Proc. IEEE Real-Time Syst. Symp.*, 2019, pp. 392–405.
- [30] Y. Kim, J. Kim, D. Chae, D. Kim, and J. Kim, "μlayer: Low latency on-device inference using cooperative single-layer acceleration and processor-friendly quantization," in *Proc. 14th EuroSys Conf.*, 2019, pp. 1–15.
- [31] L. Wang et al., "A unified optimization approach for CNN model inference on integrated GPUs," in *Proc. 48th Int. Conf. Parallel Process.*, 2019, pp. 1–10.
- [32] D. Zhang, N. Vance, Y. Zhang, M. T. Rashid, and D. Wang, "EdgeBatch: Towards AI-empowered optimal task batching in intelligent edge systems," in *Proc. IEEE Real-Time Syst. Symp.*, 2019, pp. 366–379.
- [33] W. Jang, H. Jeong, K. Kang, N. Dutt, and J.-C. Kim, "R-TOD: Real-time object detector with minimized end-to-end delay for autonomous driving," in *Proc. IEEE 41st Real-Time Syst. Symp.*, 2020, pp. 191–204.
- [34] T. L. Adam, K. M. Chandy, and J. Dickson, "A comparison of list schedules for parallel processing systems," *Commun. ACM*, vol. 17, no. 12, pp. 685–690, 1974.
- [35] J.-J. Hwang, Y.-C. Chow, F. D. Anger, and C.-Y. Lee, "Scheduling precedence graphs in systems with interprocessor communication times," *SIAM J. Comput.*, vol. 18, no. 2, pp. 244–257, 1989.
- [36] H. Kanemitsu, M. Hanada, and H. Nakazato, "Clustering-based task scheduling in a large number of heterogeneous processors," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 11, pp. 3144–3157, Nov. 2016.
- [37] M. Y. Özkaya, A. Benoit, B. Uçar, J. Herrmann, and Ü. V. Çatalyürek, "A scalable clustering-based task scheduler for homogeneous processors using DAG partitioning," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2019, pp. 155–165.
- [38] W. Liu, Z. Wang, X. Liu, N. Zeng, Y. Liu, and F. E. Alsaadi, "A survey of deep neural network architectures and their applications," *Neurocomputing*, vol. 234, pp. 11–26, 2017.
- [39] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.
- [40] J. Herrmann, J. Kho, B. Uçar, K. Kaya, and Ü. V. Çatalyürek, "Acyclic partitioning of large directed acyclic graphs," in *Proc. IEEE/ACM 17th Int. Symp. Cluster Cloud Grid Comput.*, 2017, pp. 371–380.
- [41] J. Herrmann, M. Y. Özkaya, B. Uçar, K. Kaya, and Ü. V. Çatalyürek, "Multilevel algorithms for acyclic partitioning of directed acyclic graphs," *SIAM J. Sci. Comput.*, vol. 41, no. 4, pp. A2117–A2145, 2019.
- [42] B. Hendrickson and R. W. Leland, "A multi-level algorithm for partitioning graphs," in *Proc. IEEE/ACM Conf. Supercomputing*, 1995, pp. 1–14.
- [43] M. Popp, S. Schlag, C. Schulz, and D. Seemaier, "Multilevel acyclic hypergraph partitioning," in *Proc. Proc. Workshop Algorithm Eng. Exp.*, 2021, pp. 1–15.
- [44] Z. Liu, M. Liwang, S. Hosseinalipour, H. Dai, Z. Gao, and L. Huang, "RFID: Towards low latency and reliable DAG task scheduling over dynamic vehicular clouds," *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12139–12153, Sep. 2023.

- [45] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4095–4104.
- [46] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
- [47] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.



Xiaofeng Zou received the Ph.D. degree in computer science and technology, Hunan University, Changsha, China, in 2023. His research interests include data mining, machine learning, and deep learning.



Cen Chen (Senior Member, IEEE) received the Ph.D. degree in computer science from Hunan University, Changsha, China. He is currently a Professor with the School of Future Technology, South China University of Technology, Guangzhou, China. He has authored or coauthored several research articles in international conferences and journals, such as HPCA, DAC, AAAI, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON CYBERNETICS, and many more. His research interests include parallel and distributed computing, machine learning and deep learning.



Peiying Lin received the Ph.D. degree in computer science and technology, Hunan University, Changsha, China, in 2023. Her research interests include machine learning, graph computing, and parallel computing.



Luochuan Zhang received the bachelor's degree in robotic engineering from the South China University of Technology, Guangzhou, China, in 2023. His research interests include machine learning and deep learning.



Yanwu Xu (Senior Member, IEEE) received the Ph.D. degree from the University of Science and Technology of China, Hefei, China, in 2009. He is currently a Professor with the School of Future Technology, South China University of Technology, Guangzhou, China. His research interests include ophthalmic imaging, medical image analysis, computer vision, and machine learning.



Wenjie Zhang (Member, IEEE) received the Ph.D. degree in electrical and computer engineering from the National University of Singapore (NUS), Singapore, in 2020. He is currently involved in numerous AI-oriented projects with NUS. His research interests include deep learning in smart cities and smart grids and uncertainty quantification.